

Glitch-Free Clock Multiplexer with Active Decision Logic, Symmetric Failover and Safe Recovery for Multi-Clock SoC Systems

Hai Minh Truong*, Phuong Vu Thi*, Thuan Van Le*

*Faculty of Electrical and Electronic Engineering, Phenikaa School of Engineering, Phenikaa University

Abstract—Dynamic and reliable switching among asynchronous clock sources is a persistent hurdle in advanced system-on-chip (SoC) and field-programmable gate array (FPGA) systems, where conventional clock multiplexers frequently suffer from glitches, metastability, or deadlock, especially when a clock source unexpectedly halts. This paper presents a glitch-free clock multiplexer with automatic symmetric failover, permission-driven selection logic, and cross-domain activity sensing. By combining cross-domain activity detection, permission-based arbitration, and synchronized handover logic, the design supports clean clock switching and cycle-bounded failure detection without requiring an external reference clock. Asynchronous force-disable paths further support safe transitions under the input-clock stop cases studied in this work. Simulation and FPGA prototyping on Intel Cyclone V confirm correct operation across normal operations, single- and dual-clock stops, and recovery events, with resource consumption of 14 adaptive logic modules (ALMs) and up to 515.46 MHz operating frequency. Compared to timer-based and cross-coupled designs, the proposed solution improves safety coverage with a compact footprint and frequency-dependent but cycle-bounded detection.

Index Terms—Clock multiplexer, glitch-free switching, asynchronous clocks, failover, deadlock-free, FPGA, clock domain crossing

I. INTRODUCTION

As digital systems grow in complexity and demand greater adaptability, reliable clock switching becomes a critical concern, especially in applications such as SoC platforms and FPGA-based designs [1]. Clock multiplexers, used to select between multiple clock sources for dynamic frequency scaling, power optimization, or fault recovery [2], must ensure robust and consistent operation to preserve system integrity [3].

A basic clock multiplexer can be implemented with simple combinational logic, as shown in the top of Fig. 1 [4]. The selection is governed by a digital control signal; for example, when the selector is low ($sel = 0$), the output clk_out follows the first clock input ($clk1$), and when the selector is high ($sel = 1$), it switches to the second input ($clk2$). While this approach is straightforward and area-efficient, its direct and unsynchronized nature gives rise to significant reliability hazards when switching between asynchronous clock domains.

A. Problem Definition

The conventional multiplexer has several failure modes that matter when sel and the two inputs are asynchronous.

First is the generation of glitches, which are unintended, short-lived pulses at the output. This typically occurs when sel changes while the input clocks are active, so both paths can conduct for a short overlap or a race appears in the combinational network. Downstream flip-flops may then see an extra edge and violate setup or hold timing. The circled region in Fig. 1 illustrates such a spurious spike [5]. Simple AND/OR (or NAND/NOR) mux structures are especially exposed because they do not enforce a break-before-make enable discipline.

Second, metastability can occur when asynchronous sel is sampled too close to a clock edge. With a single sampling flip-flop, the output may stay between a valid high and low for an uncertain time and then settle late to either high or low, so the mux select or enable can be wrong or delayed. A common mitigation is to pass the control signal through a two-stage flip-flop synchronizer, which greatly reduces the chance of a metastable event being released into the rest of the design [5]. Even so, eliminating metastability entirely remains difficult in high-speed systems, and the synchronizer only protects the sampled control path; it does not by itself remove output glitches caused by an unsafe combinational handoff of the clocks.

Finally, deadlock is a serious failure in which clk_out stays stuck at a fixed level and no longer follows any live source. As shown by the waveform in Fig. 2, this can happen if the selected clock halts while high and the mux enable path cannot release, so the design never switches to a live alternate even when the other input is still toggling. The system then looks frozen until the dead clock returns. Related stop cases also include a clock that halts low or stops without a clean handoff; without activity sensing and reselection, the mux alone cannot restore a usable clk_out .

B. Related Work

Prior architectures have been developed to address these issues. The break-before-make (BBM) technique [6], often implemented using cross-coupled flip-flops, improves on simpler combinational muxes that are open to both glitches and metastability during clock switching. By keeping only one clock enable active during handover, BBM prevents simultaneous clock driving and reduces metastability-related

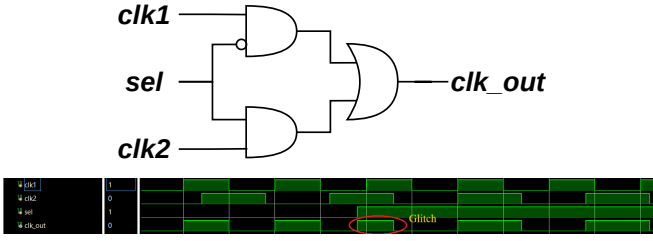


Fig. 1. Traditional AND/OR-based clock multiplexer: logic structure (top); waveform demonstrating output glitch caused by select changes during active clocks (bottom).

switching hazards [7]. In normal switching with both inputs live, this enable mutual exclusion is usually enough for a clean, glitch-free handoff and is attractive for its small area and simple control.

BBM does not cover all stop cases. In particular, if the selected clock halts while remaining high, the enable path can stay asserted and clk_out can lock high, so the mux never moves to a live source even when the other input is still toggling. Fig. 2 shows this stuck-high behavior until the faulted clock toggles again. Automatic recovery then depends on the dead clock returning, not on reselecting the live alternate. Thus BBM still leaves halt-high deadlock open and does not provide automatic failover when a live alternate remains.

Timer-based (watchdog) clock multiplexers [8] improve on BBM by using dedicated timers to detect stalled clock domains, as illustrated in Fig. 3. When inactivity is detected, they force the output to a defined safe level, typically logic low, which reduces deadlock risk when a clock halts high. If the selected clock resumes toggling after a stop, clk_out can follow the recovered source again without further user action. Compared with pure BBM, the timers therefore add stop awareness and a defined idle level at the cost of extra logic and often a lower F_{max} .

Their main limit is a passive handover policy. If the selected clock stops and sel does not change, the output stays in the safe level and does not move to a live alternate. Reselection needs a selector change or recovery of the original clock. That behavior is useful for avoiding stuck-high outputs, but it is not symmetric failover: a still-running second input is not taken unless the system or the user changes sel . The extra watchdog logic also tends to lower the maximum operating frequency relative to a minimal BBM mux.

Recent multi-clock SoCs and clock-domain crossing (CDC) paths are also a major verification concern, and pragmatic formal methods have been proposed to reduce metastability-related CDC escapes [9]. Compact glitch-free multiplexers without a third always-on reference clock remain of interest for SoC integration [10], including designs that must handle stopped or gated clocks [2], [8].

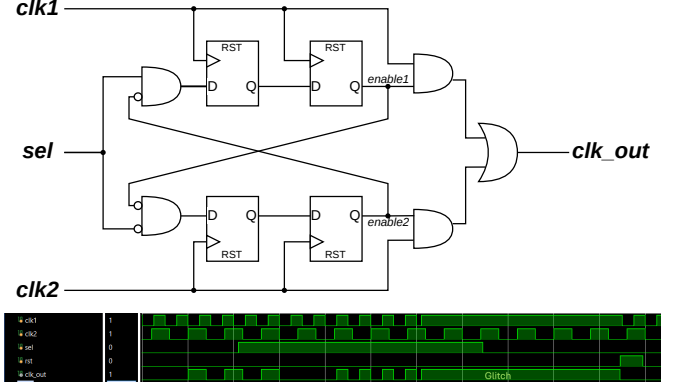


Fig. 2. Break-before-make multiplexer: cross-coupled flip-flop schematic (top); deadlock when a selected clock halts while high (bottom).

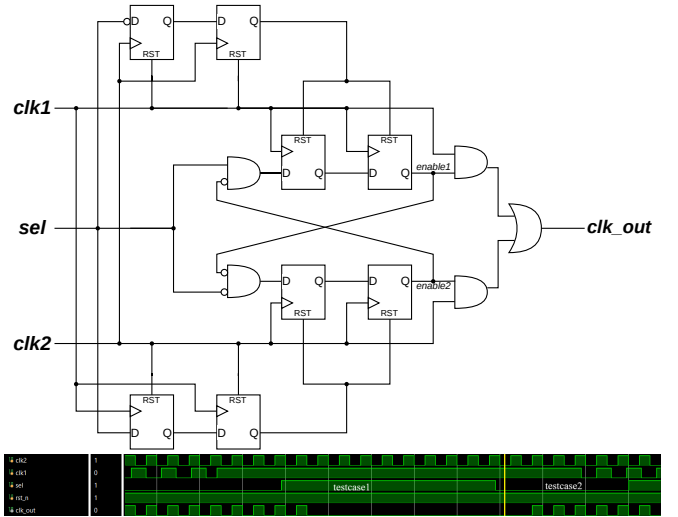


Fig. 3. Timer-based multiplexer: schematic with timers per clock source (top); waveform showing recovery upon selector change after clock failure (bottom).

C. Motivation for This Work and Paper Organization

Combinational, BBM, and timer-based muxes address different aspects of clock switching. This work targets automatic failover when either input stops and a live alternate remains, and a defined clk_out when clocks stop: force-low on staggered dual-stop via *fail*, and hold of the previous level under near-simultaneous dual-stop. The design does so without a third always-on reference clock, using layered activity sensing, decision logic, and safe handover.

The remainder of this paper is organized as follows. Section II presents the detailed three-layered design of the proposed clock multiplexer. Section III provides simulation results to verify its functional correctness and presents a comparative analysis of its FPGA implementation performance. Finally, Section IV concludes the paper and suggests

directions for future work.

II. PROPOSED ARCHITECTURE

This section presents the architecture of the proposed glitch-free clock multiplexer (GFCM). The system uses three connected layers, as shown in Fig. 4, for glitch-free switching and symmetric failover when either input stops and a live alternate remains. In a staggered dual-stop, the Activity Sensor asserts a `fail` signal when the first clock stops. That flag reaches the Safe Handover layer and can force `clk_out` low even if the second clock stops later. If both clocks stop within the same detection window, neither domain can observe the other, so no `fail` is raised and `clk_out` holds its previous level until at least one clock runs again. Table I classifies this behavior as partial dual-stop under reciprocal sensing, as shown in Figs. 6 and 7.

A. System Overview

Figure 4 displays the high-level block diagram of the proposed clock multiplexer. The circuit takes two asynchronous clock inputs, `clk1` and `clk2`, and a user selector `sel`, to produce a single, glitch-free output `clk_out`. Internally, the design is decomposed into three sequential processing layers, each with clearly delineated responsibilities.

The first layer, the Activity Sensor, monitors each input by sampling it in the other clock's domain and raises status flags when a clock has stopped. These flags go to the Decision Making layer, which combines them with `sel` to keep the user-selected source while it is live or to move to a live alternate under automatic failover. That layer only forms desired-clock signals as intent; it does not drive the output directly. The third layer, Safe Handover, turns that intent into output enables with synchronized enable registers and mutual exclusion for break-before-make, glitch-free switching.

This modular decomposition, as visually depicted in Fig. 4, clarifies both the dataflow and the control dependencies of the system, and forms the basis for ease of analysis and future extensibility. Signal names in Figs. 4 and 5 match the register-transfer level (RTL) netlist including: `fail1/fail2`, `desired_clk1`, `desired_clk2`, `enable1`, `enable2`, `clk_out`.

B. Layer 1: Activity Sensor

The Activity Sensor layer interfaces directly with the two asynchronous clock domains. Utilizing cross-domain sampling, each clock is sampled in the other's domain, and transitions are detected by XORing consecutive samples [8]. A saturating synchronous counter, running in the opposing clock domain, accumulates the number of cycles since the last detected transition. If this counter exceeds a predefined threshold, a failure flag `fail1` or `fail2` is asserted for that clock. The threshold is set to three cycles of the observing clock for the frequency pairs evaluated in this work. This value covers the synchronizer and edge-detection delay while avoiding false failure declarations caused by short sampling gaps. In general, the threshold must exceed the maximum

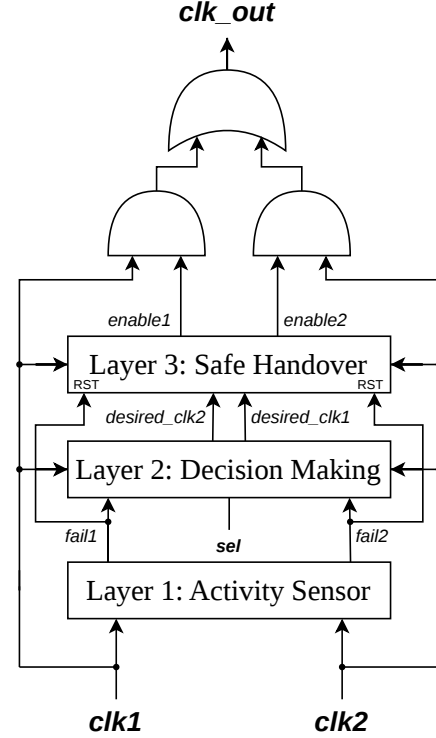


Fig. 4. Block diagram of the proposed clock multiplexer. Layer 1 reports `fail1/fail2`; Layer 2 forms `desired_clk1/desired_clk2`; Layer 3 drives `enable1/enable2` to produce `clk_out`.

number of observing-clock cycles expected between two captured transitions of the monitored clock, including the synchronizer uncertainty. If both clocks stop at once, both counters stop and neither `fail` can assert, because health is only observed from a still-active domain.

C. Layer 2: Decision Making

The Decision Making layer is responsible for the real-time assessment and selection of the appropriate clock source. Its operation is governed by a clear policy that uses the status from Layer 1 (`fail1`, `fail2`) and the user's selection preference (`sel`) to determine the system's intent.

The policy prioritizes the clock chosen by the user while that clock is still live. If the selected clock stops while the alternate remains active, the logic overrides the user's choice and designates the live alternate as the desired source, which is automatic failover. If both clocks stop in a staggered dual-stop and at least one `fail` is raised, this layer de-asserts both desired-clock signals so Layer 3 can force a low output. That safe command still depends on receiving at least one `fail` from Layer 1. Under near-simultaneous dual-stop, when no `fail` is generated, this layer does not change its outputs. In all modes, Layer 2 only produces the desired-clock signals

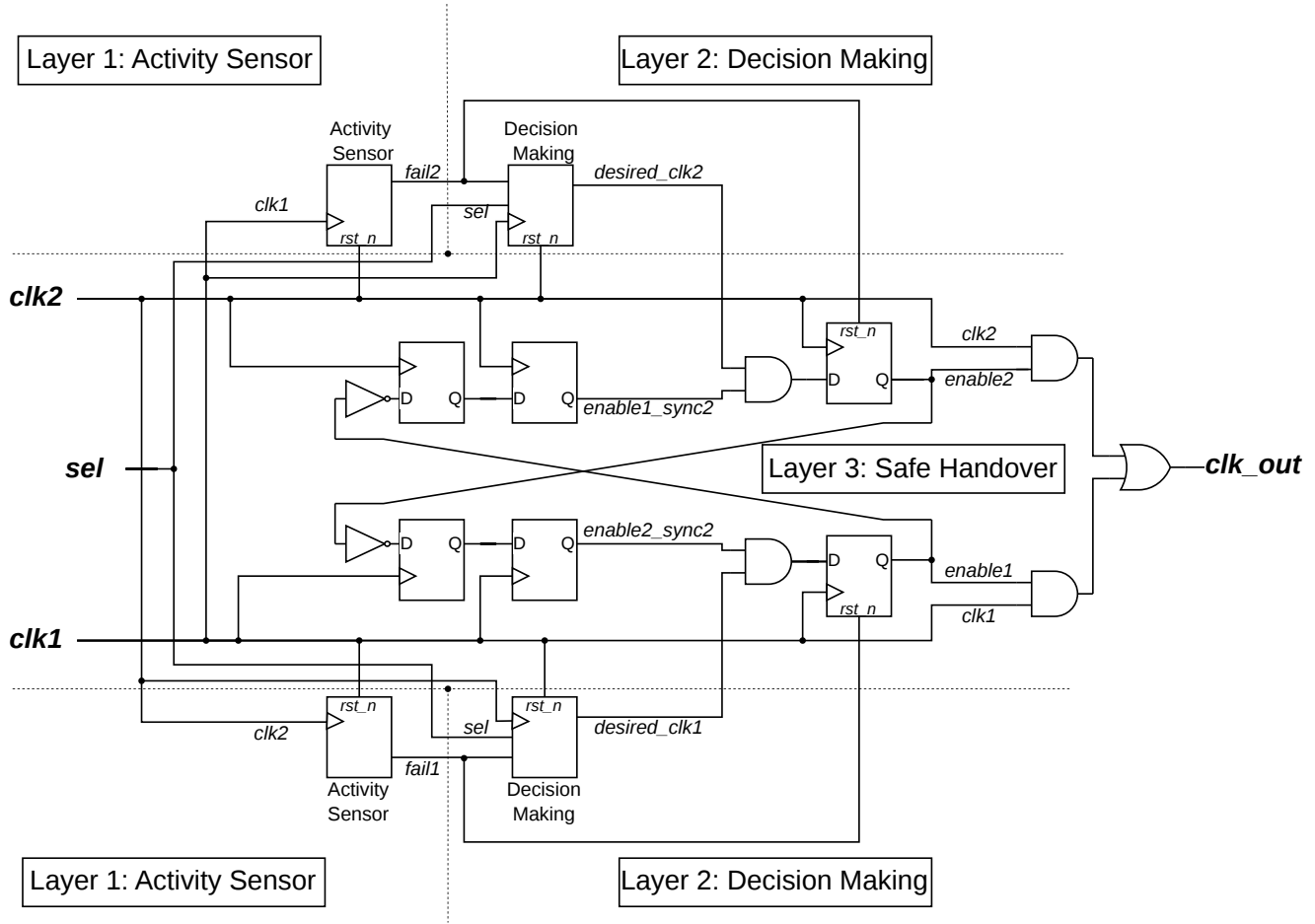


Fig. 5. Schematic of the proposed dual-clock GFCM (read top/bottom as the two domains). Left of the dotted line: Layer 1 activity sensors produce $fail1/fail2$. Center: Layer 2 decision blocks combine those flags with sel into $desired_clk1/desired_clk2$. Right (Layer 3): enable signals are synchronized across domains, forced mutually exclusive (break-before-make), then AND-gated with $clk1/clk2$ and OR-combined to form clk_out . Crossed wires between domains carry the synchronized enables used for mutual exclusion.

that guide the final handover logic.

D. Layer 3: Safe Handover

The final layer, Safe Handover, implements the clock selection decided by the Decision Making layer. Its role is glitch-free, mutually exclusive switching between the two sources so that only one enable is active at a time and enable changes do not create short output spikes. The high-level arrangement is shown in Fig. 4, and the detailed paths appear in Fig. 5.

First, the desired-clock signals from Layer 2 are passed through dedicated flip-flop stages in their respective clock domains. These stages align each enable to its own clock, pipeline the control path, and keep enable updates on stable edges. That reduces the hazards of gating a free-running clock with purely combinational control.

Second, mutual exclusion logic keeps both output enables from being active at the same time. This break-before-make rule is what gives a clean handoff when the design switches between sources, including during automatic failover when a live alternate is selected.

Third, asynchronous force-disable and output gating can drive clk_out low when Layer 2 has de-asserted both desired-clock signals, which is the staggered dual-stop case after at least one $fail$ has been raised, or when reset is asserted. This force-low path is the safe-state action for those conditions. It is separate from near-simultaneous dual-stop: if Layer 1 raises no $fail$ and Layer 2 does not change intent, Layer 3 receives no new command to force low, so clk_out can hold its previous level under reciprocal sensing, as described at the start of Section II.

In summary, the architecture provides glitch-free handover, automatic failover, and safe staggered dual-stop behavior

under the reciprocal-sensing assumptions above. The high-level structure is shown in Fig. 4.

To provide deeper insight into the practical realization of these mechanisms, the complete logic is detailed in the accompanying schematic diagram, as depicted in Fig. 5. This schematic reveals the precise signal paths, flip-flop stages, and gating elements that implement each architectural function, clarifying how abstract principles are mapped to concrete hardware.

III. RESULTS AND EVALUATION

A. Simulation and Functional Verification

To validate the correctness and reliability of the proposed design, functional verification was performed. The architecture was modeled in Verilog, and a dedicated testbench was developed to generate the stimulus for a range of stop and recovery scenarios. This testbench instantiates the proposed clock multiplexer as the Design Under Test (DUT) and uses procedural blocks to control the behavior of the input clocks, selector, and reset signals. The resulting waveforms confirm failover and recovery behavior for the cases studied in this work.

The simulation waveform in Fig. 6 shows multiple stop and recovery scenarios. The first circled region highlights the corner case where `clk1` stops, followed by `clk2` stopping shortly thereafter. Because the time between the two stops is less than the detection threshold, the system does not assert a `fail` signal and the output (`clk_out`) holds its previous level. The second circled region shows automatic failover: when the selected clock (`clk1`) stops while the live alternate (`clk2`) remains active, the output switches to track `clk2`. Finally, the third circled region demonstrates recovery, where the output returns to tracking the user-selected clock (`clk1`) as soon as it resumes toggling after its period of inactivity. This single waveform covers hold under near-simultaneous dual-stop, symmetric failover to a live alternate, and recovery.

Figure 7 presents the staggered dual-stop case. Here, the selected clock (`clk2`) stops first. There is a sufficient time delay before the alternate clock (`clk1`) also stops, allowing the Activity Sensor in the `clk1` domain to detect the stop of `clk2`. This triggers a `fail2` signal, which commands the Decision Making layer to de-assert all desired-clock signals. As shown in the circled region, this action forces the output `clk_out` to logic low before the second clock also stops. This staggered dual-stop force-low path is distinct from near-simultaneous dual-stop, where the output holds its previous level, as described in Section II.

Collectively, the simulations confirm automatic failover and recovery, safe force-low behavior during staggered dual-stop events, and stable hold-state behavior when both clocks stop within the same detection window.

B. FPGA Implementation and Resource Utilization

To evaluate performance and cost, all architectures were designed in Verilog HDL and synthesized for an Intel Cyclone V 5CEFA2F23C8 FPGA. The measured resource util-

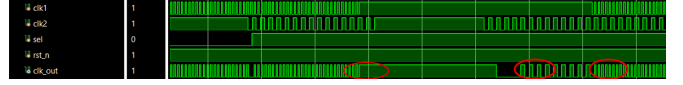


Fig. 6. Simulation of near-simultaneous dual-stop hold (first circle), symmetric failover to the live alternate (second circle), and recovery of the user-selected clock (third circle).



Fig. 7. Simulation of staggered dual-stop. After the first clock stops, `clk_out` is forced low before the second clock also stops.

ization, including Adaptive Logic Modules (ALMs), Logic Array Blocks (LABs), Adaptive Look-Up Tables (ALUTs), and Dedicated Logic Registers (DLRs)¹, is summarized in Table I. This allows a direct comparison under consistent tool and device conditions.

Table I reports area, maximum frequency, and safety-related features for the four designs. The conventional BBM architecture [6] is the most area-efficient at 3 ALMs, but it offers little beyond glitch-free switching and does not recover on its own when the selected clock stops high. The timer-based approach [8] adds deadlock-free behavior by forcing a safe level when activity is lost, at the cost of 12 ALMs and a lower maximum frequency; it still does not hand over to an alternate live clock unless the selector changes or the original clock returns. The clock-reference-based design [10] supports full dual-stop safety, but it depends on a third continuously running reference clock and consumes 27 ALMs. In contrast, the proposed GFCM reaches automatic failover and safe staggered dual-stop handling without a third reference clock at 14 ALMs and 515.46 MHz; dual-stop stays partial under reciprocal sensing, as defined in Section II and Table I.

At the module level, Layer 1 holds most of the registers in the cross-domain samplers and saturating activity counters; Layer 2 is light combinational arbitration of `sel` and the `fail` flags; Layer 3 holds enable synchronizers, mutual exclusion, and output gating. With two clocks, each domain monitors the other: a live clock is the only observer, so a single stop can still be detected, while a near-simultaneous dual stop leaves no observer. If the architecture is extended to more than two inputs, each additional source requires a corresponding sensing and gating path, while any active source can serve as an observing domain for the halted clocks. For an N -source implementation with one sensing and gating path per source, the datapath complexity grows approximately linearly with N , whereas the arbitration complexity depends on the adopted priority or health-aware selection policy. Full pairwise reciprocal monitoring would, however, increase

¹These terms refer to key FPGA resources: ALUTs are the basic combinational logic elements; DLRs are the flip-flops; ALMs are the fundamental building blocks containing both ALUTs and DLRs; and LABs are clusters of multiple ALMs.

Table I: Feature and Implementation Comparison for Various Clock Multiplexer Designs.

Design	Glitch-Free	Ref. Clock	Deadlock-Free	Dual-Stop Safety	Latency Characteristic	ALMs	LABs	ALUTs	DLRs	Max Freq. (MHz)
Break-before-make [5], [6]	Yes	No	No	No	Input-clock-cycle bounded	3	2	4	6	580.05
Timer-based [8]	Yes	No	Yes	No	Frequency-ratio dependent	12	4	5	10	367.92
Clock-ref-based [10]	Yes	Yes	Yes	Yes	Reference-cycle bounded	27	4	51	41	352.61
Proposed GFCM	Yes	No	Yes	Partial*	Observer-clock-cycle bounded	14	3	24	22	515.46

* Staggered dual-stop can force `clk_out` low via `fail`; near-simultaneous dual-stop holds the previous level (Partial under reciprocal sensing).

the sensing complexity more rapidly and may require a hierarchical or ring-based organization for larger clock sets.

The structural Verilog implementation requires no vendor-specific phase-locked loop (PLL) and no additional always-on reference clock; integration mainly needs standard reset distribution and CDC constraints on the reciprocal sampling paths.

IV. CONCLUSION

This paper has presented a glitch-free clock multiplexer (GFCM) for asynchronous sources that combines reciprocal activity sensing, decision logic, and break-before-make handover without an external reference clock. The design supports automatic failover when one input clock stops and the other remains active, and safe staggered dual-stop force-low once a `fail` is raised. Dual-stop coverage remains partial under reciprocal sensing: if both clocks stop in the same detection window, no `fail` is raised and the output holds its previous level.

Simulation confirmed correct behavior for single-clock stop, recovery, staggered dual-stop, and near-simultaneous dual-stop cases. On an Intel Cyclone V FPGA the design uses 14 ALMs and reaches 515.46 MHz, between a minimal BBM baseline and heavier timer- or reference-based options in Table I, with frequency-dependent but cycle-bounded stop detection.

The architecture can be applied in multi-clock SoCs that switch or recover among oscillators under power gating or domain stop while keeping a defined `clk_out`. Future work includes multi-way multiplexing, where additional live inputs can serve as observing domains, and adaptive thresholds for failure detection with lighter monitoring logic.

REFERENCES

- [1] T. Xanthopoulos, *Clocking in Modern VLSI Systems*, ser. Integrated Circuits and Systems. Springer US, 2009.
- [2] E. Kyriakakis, K. Tange, N. Reusch, E. O. Zaballa, X. Fafoutis, M. Schoeberl, and N. Dragoni, "Fault-tolerant clock synchronization using precise time protocol multi-domain aggregation," in *2021 IEEE 24th International Symposium on Real-Time Distributed Computing (ISORC)*, 2021, pp. 114–122.
- [3] K. Vipin and S. A. Fahmy, "Fpga dynamic and partial reconfiguration: A survey of architectures, methods, and applications," *ACM Computing Surveys (CSUR)*, vol. 51, no. 4, pp. 1–39, 2018.
- [4] J. Wakerly, *Digital Design Principles And Practices 4Th Ed.* Prentice-Hall Of India Pvt. Limited, 2006.
- [5] R. Ginosar, "Fourteen ways to fool your synchronizer," in *Ninth International Symposium on Asynchronous Circuits and Systems, 2003. Proceedings.*, 2003, pp. 89–96.
- [6] D. Harris and S. Harris, *Digital Design and Computer Architecture*. Morgan Kaufmann, 2012.
- [7] K. R. Talupuru and S. Athi, "Achieving glitch-free clock domain crossing signals using formal verification, static timing analysis, and sequential equivalence checking," in *2011 12th International Workshop on Microprocessor Test and Verification*, 2011, pp. 5–9.
- [8] S. Zeidler, O. Schrape, A. Breitenreiter, and M. Krstić, "A glitch-free clock multiplexer for non-continuously running clocks," in *2020 23rd Euromicro Conference on Digital System Design (DSD)*, 2020, pp. 11–15.
- [9] A. Kumar, M. U. H. Khan, and B. Mittra, "Pragmatic formal verification methodology for clock domain crossing (cdc)," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06533>
- [10] T. V. Le and N. Van Dinh, "Glitch-free clock multiplexer using reference-clock missing-pulse detection and safe-off recovery for SoC applications," in *2025 International Conference on Engineering and Emerging Technologies (ICEET)*, 2025, pp. 1–6.